

Smoothing Spline ANOVA

Nathaniel E. Helwig

Assistant Professor of Psychology and Statistics
University of Minnesota (Twin Cities)



Updated 04-Jan-2017

Copyright © 2017 by Nathaniel E. Helwig

Outline of Notes

1) Introduction

- Parametric regression
- Nonparametric regression
- Smoothing splines

2) Background Theory

- Averaging operators
- Hilbert spaces
- Reproducing kernels

3) Estimation & Inference:

- Penalized least squares
- Smoothing parameter selection
- Bayesian confidence intervals

4) SSANOVA in Practice:

- One-way SSANOVA
- Two-way SSANOVA (additive)
- Two-way SSANOVA (interactive)

For a thorough treatment see:

Gu, C. (2013). *Smoothing spline ANOVA models, 2nd edition*. New York: Springer-Verlag.

Introduction

Parametric Regression Model: Scalar Form

The multiple linear regression model has the form

$$y_i = \sum_{j=1}^p b_j x_{ij} + e_i$$

for $i \in \{1, \dots, n\}$ where

- $y_i \in \mathbb{R}$ is the real-valued response for the i -th observation
- $b_j \in \mathbb{R}$ is the j -th predictor's regression slope
- $x_{ij} \in \mathbb{R}$ is the j -th predictor for the i -th observation
- $e_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$ is Gaussian measurement error

Implies that $(y_i | x_{i1}, \dots, x_{ip}) \stackrel{\text{ind}}{\sim} \mathcal{N}(\sum_{j=1}^p b_j x_{ij}, \sigma^2)$

Parametric Regression Model: Matrix Form

The multiple linear regression model has the form

$$\mathbf{y} = \mathbf{X}\mathbf{b} + \mathbf{e}$$

where

- $\mathbf{y} = (y_1, \dots, y_n)' \in \mathbb{R}^n$ is the $n \times 1$ response vector
- $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_p] \in \mathbb{R}^{n \times p}$ is the $n \times p$ design matrix
 - $\mathbf{x}_j = (x_{1j}, \dots, x_{nj})' \in \mathbb{R}^n$ is j -th predictor vector ($n \times 1$)
- $\mathbf{b} = (b_1, \dots, b_p)' \in \mathbb{R}^p$ is $p \times 1$ vector of coefficients
- $\mathbf{e} = (e_1, \dots, e_n)' \in \mathbb{R}^n$ is the $n \times 1$ error vector

Implies that $(\mathbf{y}|\mathbf{x}) \sim N(\mathbf{X}\mathbf{b}, \sigma^2\mathbf{I}_n)$

Ordinary Least Squares Solution

The ordinary least squares (OLS) problem is

$$\min_{\mathbf{b} \in \mathbb{R}^p} \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{b}\|^2 \quad \longleftrightarrow \quad \min_{\mathbf{b} \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where $\|\cdot\|$ denotes the Euclidean norm and $\hat{y}_i = \sum_{j=1}^p b_j x_{ij}$.

The OLS solution has the form

$$\hat{\mathbf{b}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$$

and the fitted values corresponding to $\hat{\mathbf{b}}$ are given by

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{b}} = \mathbf{H}\mathbf{y}$$

where $\mathbf{H} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'$ is the hat matrix.

Summary of Results

Using the model assumption $(\mathbf{y}|\mathbf{x}) \sim N(\mathbf{X}\mathbf{b}, \sigma^2\mathbf{I}_n)$, we have

$$\hat{\mathbf{b}} \sim N(\mathbf{b}, \sigma^2(\mathbf{X}'\mathbf{X})^{-1})$$

$$\hat{\mathbf{y}} \sim N(\mathbf{X}\mathbf{b}, \sigma^2\mathbf{H})$$

$$\hat{\mathbf{e}} \sim N(\mathbf{0}, \sigma^2(\mathbf{I}_n - \mathbf{H}))$$

where $\hat{\mathbf{e}} = \mathbf{y} - \hat{\mathbf{y}}$ is the residual vector.

Typically σ^2 is unknown, so we use the MSE $\hat{\sigma}^2 = \frac{1}{n-p} \sum_{i=1}^n \hat{e}_i^2$.

Nonparametric Regression Model

The Gaussian nonparametric regression model has the form

$$y_i = \eta(\mathbf{x}_i) + e_i$$

for $i \in \{1, \dots, n\}$ where

- $y_i \in \mathbb{R}$ is the real-valued response for the i -th observation
- $\mathbf{x}_i \in \mathbb{R}^p$ is the predictor vector for the i -th observation
- $\eta : \mathbb{R}^p \rightarrow \mathbb{R}$ is an unknown smooth function
- $e_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$ is Gaussian measurement error

Implies that $(y_i | x_{i1}, \dots, x_{ip}) \stackrel{\text{ind}}{\sim} \mathcal{N}(\eta(\mathbf{x}_i), \sigma^2)$

Additive versus Interactive Models

Suppose that $\mathbf{x}_i = (x_{i1}, x_{i2})$ with $x_{i1} \in \mathcal{X}_1$ and $x_{i2} \in \mathcal{X}_2$.

We could fit one of two possible models:

$$\textit{Additive} : \eta(\mathbf{x}_i) = \eta_0 + \eta_1(x_{i1}) + \eta_2(x_{i2})$$

$$\textit{Interaction} : \eta(\mathbf{x}_i) = \eta_0 + \eta_1(x_{i1}) + \eta_2(x_{i2}) + \eta_{12}(x_{i1}, x_{i2})$$

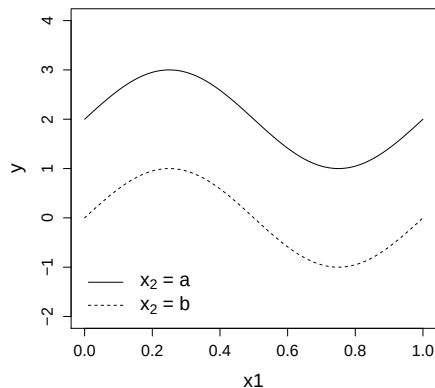
where

- η_0 is a constant function
- η_1 is main effect of first predictor
- η_2 is main effect of second predictor
- η_{12} is interaction effect

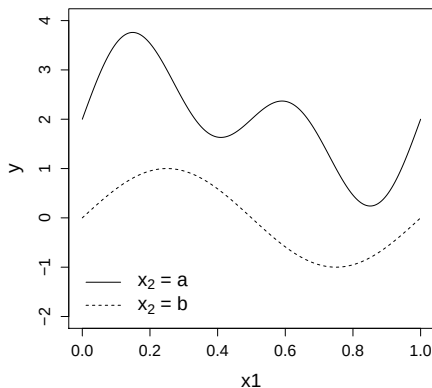
Example 1: Continuous and Nominal Covariates

$\mathbf{x}_i = (x_{i1}, x_{i2})$ with $x_{i1} \in [0, 1]$ and $x_{i2} \in \{a, b\}$.

Additive



Interaction



Example 1: R Code

```
addfun = function(x1,x2){
  funval = sin(2*pi*x1)
  idx = which(x2=="a")
  funval[idx] = funval[idx] + 2
  funval
}

intfun = function(x1,x2){
  funval = sin(2*pi*x1)
  idx = which(x2=="a")
  funval[idx] = funval[idx] + 2 + sin(4*pi*x1[idx])
  funval
}

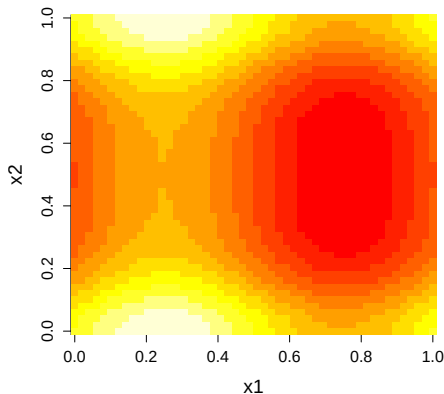
dev.new(width=12,height=6,noRStudioGD=TRUE)
par(mfrow=c(1,2))
x1 = seq(0,1,length=200)
plot(x1,addfun(x1,rep("a",200)),type="l",ylim=c(-2,4),main="Additive",
      ylab="y",cex.axis=1.25,cex.lab=1.5,cex.main=3)
lines(x1,addfun(x1,rep("b",200)),lty=2)
legend("bottomleft",legend=c(expression(x[2]*" = "*a),expression(x[2]*" = "*b)),
      lty=1:2,bty="n",cex=1.5)

plot(x1,intfun(x1,rep("a",200)),type="l",ylim=c(-2,4),main="Interaction",
      ylab="y",cex.axis=1.25,cex.lab=1.5,cex.main=3)
lines(x1,intfun(x1,rep("b",200)),lty=2)
legend("bottomleft",legend=c(expression(x[2]*" = "*a),expression(x[2]*" = "*b)),
      lty=1:2,bty="n",cex=1.5)
```

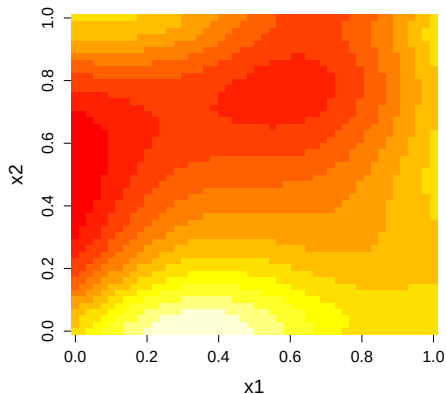
Example 2: Two Continuous Covariates

$\mathbf{x}_i = (x_{i1}, x_{i2})$ with $x_{i1}, x_{i2} \in [0, 1]$.

Additive



Interaction



Example 2: R Code

```
addfun = function(x1,x2){  
  sin(2*pi*x1) + cos(4*pi*x2*(1-x2))  
}  
intfun = function(x1,x2){  
  sin(2*pi*x1) + cos(4*pi*x2*(1-x2)) + 2*sin(pi*(x1-x2))  
}  
  
xs = seq(0,1,length=50)  
xg = expand.grid(xs,xs)  
dev.new(width=12,height=6,noRStudioGD=TRUE)  
par(mfrow=c(1,2))  
zmat = matrix(addfun(xg[,1],xg[,2]),50,50)  
image(xs,xs,zmat,xlab="x1",ylab="x2",main="Additive",  
       cex.axis=1.25,cex.lab=1.5,cex.main=3)  
zmat = matrix(intfun(xg[,1],xg[,2]),50,50)  
image(xs,xs,zmat,xlab="x1",ylab="x2",main="Interaction",  
       cex.axis=1.25,cex.lab=1.5,cex.main=3)
```

Smoothing Splines on $\{1, \dots, K\}$

Suppose $x_i \in \{1, \dots, K\}$ and note that $\eta_{\mathbf{f}}$ is a vector of length K .

- $\mathbf{f} = (f_1, \dots, f_K)' \in \mathbb{R}^K$ is vector corresponding to $\eta_{\mathbf{f}}$
- $\eta_{\mathbf{f}}(1) = f_1, \eta_{\mathbf{f}}(2) = f_2, \dots, \eta_{\mathbf{f}}(K) = f_K$
- Let $\bar{\eta}_{\mathbf{f}} = \sum_{x=1}^K \eta_{\mathbf{f}}(x)/K$ denote the mean

A **nominal smoothing spline** is the $\eta_{\lambda} \in \mathbb{R}^K$ that minimizes

$$\frac{1}{n} \sum_{i=1}^n (y_i - \eta_{\mathbf{f}}(x_i))^2 + \lambda J(\eta_{\mathbf{f}})$$

where $\lambda \geq 0$ is **smoothing parameter** and $J(\eta_{\mathbf{f}})$ is **roughness penalty**.

- $J(\eta_{\mathbf{f}}) = \sum_{x=1}^K (\eta_{\mathbf{f}}(x) - \bar{\eta}_{\mathbf{f}})^2$ to shrink towards constant
- $J(\eta_{\mathbf{f}}) = \sum_{x=1}^K \eta_{\mathbf{f}}(x)^2$ to shrink towards zero

Polynomial Smoothing Splines on $[0,1]$

Suppose $x_i \in [0, 1]$ and let $\mathcal{C}^{(m)}[0, 1] = \{\eta : \eta^{(m)} \in \mathcal{L}_2[0, 1]\}$.

- $\eta^{(m)} = \frac{d^m \eta}{dx^m}$ denotes m -th derivative of η
- $\mathcal{L}_2[0, 1] = \{\eta : \int_0^1 \eta^2 dx < \infty\}$

A **polynomial smoothing spline** is the $\eta_\lambda \in \mathcal{C}^{(m)}[0, 1]$ that minimizes

$$\frac{1}{n} \sum_{i=1}^n (y_i - \eta(x_i))^2 + \lambda \int_0^1 (\eta^{(m)})^2 dx$$

where $\lambda \geq 0$ is the **smoothing parameter** and m is **spline order**.

- Related to **natural spline** in numerical analysis literature

Cubic Smoothing Splines

Setting $m = 2$ results in classic **cubic smoothing spline**.

- $\check{x}_1 < \check{x}_2 < \dots < \check{x}_q$ are “knots” (distinct x_i values)
- η_λ is piecewise cubic polynomial, and is linear beyond $\check{x}_1$ and $\check{x}_q$
- η_λ is three-times differentiable, and 3rd derivative jumps at “knots”
- As $\lambda \rightarrow 0$, η_λ approaches minimum curvature interpolant
- As $\lambda \rightarrow \infty$, η_λ approaches simple linear regression

Can also view cubic smoothing spline as solution to

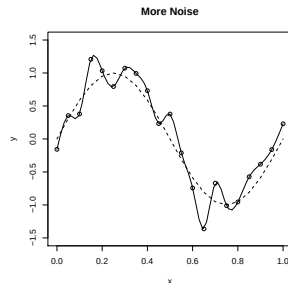
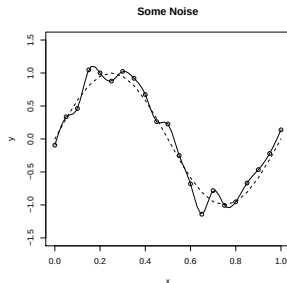
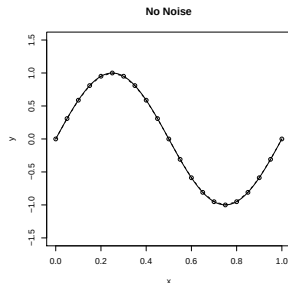
$$\min \frac{1}{n} \sum_{i=1}^n (y_i - \eta(x_i))^2 \quad \text{subject to} \quad \int_0^1 \ddot{\eta}^2 dx \leq \rho$$

for some $\rho \geq 0$, which is least-squares with soft constraint.

Example with R's `spline` Function

$y_i = \sin(2\pi x_i) + e_i$ where $x_i = i/20$ for $i \in \{0, 1, 2, \dots, 20\}$ and

- No Noise: $e_i = 0 \forall i$
- Some Noise: $e_i \stackrel{\text{iid}}{\sim} N(0, 0.15^2)$
- More Noise: $e_i \stackrel{\text{iid}}{\sim} N(0, 0.25^2)$



spline Function (R code)

```
dev.new(width=12,height=4,noRStudioGD=TRUE)
par(mfrow=c(1,3))
x = seq(0,1,length=21)
y = sin(2*pi*x)
mysp = spline(x,y,method="natural")
plot(x,y,main="No Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(mysp)
```

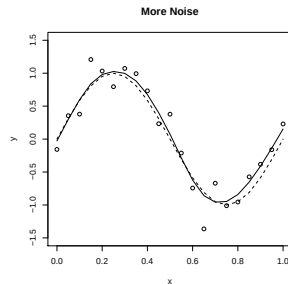
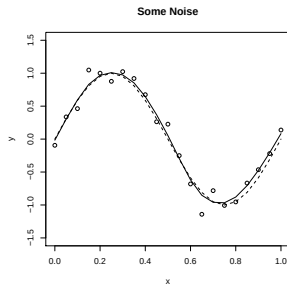
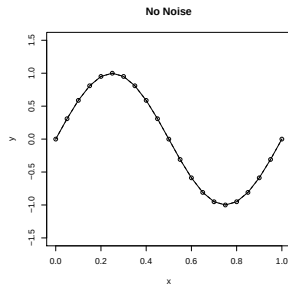
```
set.seed(1)
x = seq(0,1,length=21)
y = sin(2*pi*x) + rnorm(21,sd=0.15)
mysp = spline(x,y,method="natural")
plot(x,y,main="Some Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(mysp)
```

```
set.seed(1)
x = seq(0,1,length=21)
y = sin(2*pi*x) + rnorm(21,sd=0.25)
mysp = spline(x,y,method="natural")
plot(x,y,main="More Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(mysp)
```

Same Example with R's `smooth.spline` Function

$y_i = \sin(2\pi x_i) + e_i$ where $x_i = i/20$ for $i \in \{0, 1, 2, \dots, 20\}$ and

- No Noise: $e_i = 0 \forall i$
- Some Noise: $e_i \stackrel{\text{iid}}{\sim} N(0, 0.15^2)$
- More Noise: $e_i \stackrel{\text{iid}}{\sim} N(0, 0.25^2)$



smooth.spline Function (R code)

```
dev.new(width=12,height=4,noRStudioGD=TRUE)
par(mfrow=c(1,3))
set.seed(1)
x = seq(0,1,length=21)
y = sin(2*pi*x)
mysp = smooth.spline(x,y)
plot(x,y,main="No Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(x,mysp$y)
```

```
set.seed(1)
x = seq(0,1,length=21)
y = sin(2*pi*x) + rnorm(21,sd=0.15)
mysp = smooth.spline(x,y)
plot(x,y,main="Some Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(x,mysp$y)
```

```
set.seed(1)
x = seq(0,1,length=21)
y = sin(2*pi*x) + rnorm(21,sd=0.25)
mysp = smooth.spline(x,y)
plot(x,y,main="More Noise",ylim=c(-1.5,1.5))
lines(x,sin(2*pi*x),lty=2)
lines(x,mysp$y)
```

Background Theory

One-Way ANOVA Decomposition

Consider the standard one-way ANOVA model

$$y_{ij} = \mu_j + e_{ij}$$

for $i \in \{1, \dots, n_j\}$ and $j \in \{1, \dots, K\}$.

Typically, we want to decompose the treatment effects such as

$$\mu_j = \mu + \alpha_j$$

where μ is overall mean and α_j is treatment effect such that

- $\alpha_1 = 0$ if first group is control
- $\sum_{j=1}^K \alpha_j = 0$ if using effect coding

One-Way ANOVA and Averaging Operators

Consider the standard one-way ANOVA model using a smoothing spline on $x_i \in \{1, \dots, K\}$

$$y_i = \eta(x_i) + \epsilon_i$$

for $i \in \{1, \dots, n\}$ where $n = \sum_{j=1}^K n_j$.

The ANOVA decomposition $\mu_j = \mu + \alpha_j$ can be written as

$$\eta = A\eta + (I - A)\eta = \eta_0 + \eta_c$$

where A “averages out” η to return a constant η_0 .

- $\alpha_1 = 0$ corresponds to $A\eta = \eta(1)$
- $\sum_{j=1}^K \alpha_j = 0$ corresponds to $A\eta = \sum_{x=1}^K \eta(x)/K$

Averaging Operators on Continuous Domain

For a continuous domain $\mathcal{X} = [a, b]$ we can decompose η such as

$$\eta = A\eta + (I - A)\eta = \eta_0 + \eta_c$$

where A “averages out” η to return a constant η_0 .

- Need **averaging operator** A defined such that $A(A\eta) = A\eta = \eta_0$
- Need **identity operator** I defined such that $I\eta = \eta$

Note that η_0 is overall constant, and η_c is treatment (contrast) effect.

For a function defined on $\mathcal{X} = [0, 1]$, we could define

- $A\eta = \eta(0)$
- $A\eta = \int_0^1 \eta(z)dz$

Two-Way ANOVA Decomposition

Consider the standard two-way ANOVA model

$$y_{ijk} = \mu_{jk} + e_{ijk}$$

for $i \in \{1, \dots, n_{jk}\}$, $j \in \{1, \dots, a\}$, and $k \in \{1, \dots, b\}$.

Typically, we want to decompose the treatment effects such as

$$\mu_{jk} = \mu + \alpha_j + \beta_k + \gamma_{jk}$$

where μ is overall mean and

- α_j is main effect of Factor A such that $\sum_{j=1}^a \alpha_j = 0$
- β_k is main effect of Factor B such that $\sum_{k=1}^b \beta_k = 0$
- γ_{jk} is interaction effect such that $\sum_{j=1}^a \gamma_{jk} = \sum_{k=1}^b \gamma_{jk} = 0 \forall j, k$

Two-Way ANOVA and Averaging Operators

Consider the standard two-way ANOVA model using a smoothing spline on $\mathbf{x}_i = (x_{i1}, x_{i2}) \in \mathcal{X}_1 \times \mathcal{X}_2 = \{1, \dots, a\} \times \{1, \dots, b\}$

$$y_i = \eta(\mathbf{x}_i) + \epsilon_i$$

for $i \in \{1, \dots, n\}$ where $n = \sum_{j=1}^a \sum_{k=1}^b n_{jk}$.

The ANOVA decomposition $\mu_{jk} = \mu + \alpha_j + \beta_k + \gamma_{jk}$ can be written as

$$\begin{aligned} \eta &= [A_{\mathcal{X}_1} + (I - A_{\mathcal{X}_1})][A_{\mathcal{X}_2} + (I - A_{\mathcal{X}_2})]\eta \\ &= \underbrace{A_{\mathcal{X}_1} A_{\mathcal{X}_2} \eta}_{\eta_0} + \underbrace{(I - A_{\mathcal{X}_1}) A_{\mathcal{X}_2} \eta}_{\eta_1} + \underbrace{A_{\mathcal{X}_1} (I - A_{\mathcal{X}_2}) \eta}_{\eta_2} + \underbrace{(I - A_{\mathcal{X}_1}) (I - A_{\mathcal{X}_2}) \eta}_{\eta_{12}} \end{aligned}$$

where $A_{\mathcal{X}_1}$ and $A_{\mathcal{X}_2}$ are averaging operators such that

- $A_{\mathcal{X}_1}(A_{\mathcal{X}_1}\eta) = A_{\mathcal{X}_1}\eta$ is constant for all $x_{i1} \in \mathcal{X}_1$
- $A_{\mathcal{X}_2}(A_{\mathcal{X}_2}\eta) = A_{\mathcal{X}_2}\eta$ is constant for all $x_{i2} \in \mathcal{X}_2$

Linear Spaces and Functionals

Suppose that $\eta, \phi \in \mathcal{L}$ where the set $\mathcal{L}$ satisfies:

- $\eta + \phi \in \mathcal{L}$
- $a\mathcal{L} \in \mathcal{L}$ for any scalar a

If these two conditions are met, we say that $\mathcal{L}$ is a **linear space**.

A **functional** L in $\mathcal{L}$ operates on $\eta \in \mathcal{L}$ and returns a real number.

- **Linear functional:** $L(\eta + \phi) = L\eta + L\phi$ and $L(a\eta) = aL\eta$
- **Bilinear functional:** linear functional of two variables
 - $J(a\eta + b\phi, \psi) = aJ(\eta, \psi) + bJ(\phi, \psi)$
 - $J(\eta, a\phi + b\psi) = aJ(\eta, \phi) + bJ(\eta, \psi)$
- **Symmetry:** $J(\eta, \phi) = J(\phi, \eta)$ for all $\eta, \phi \in \mathcal{L}$
- **Positive definite:** $J(\eta) = J(\eta, \eta) > 0$ for all $\eta \in \mathcal{L}$
- **Non-negative definite:** $J(\eta) = J(\eta, \eta) \geq 0$ for all $\eta \in \mathcal{L}$
- **Quadratic:** bilinear, symmetric, and non-negative definite

Inner Products and Norms

In a linear space $\mathcal{L}$, an **inner-product** is a positive definite bilinear form.

- We will use the notation $\langle \cdot, \cdot \rangle$ to denote an inner-product.

The inner-product defines a **norm** in $\mathcal{L}$, which provides a metric to measure distance between two objects $\eta, \phi \in \mathcal{L}$.

- We will use the notation $\|\eta\| = \sqrt{\langle \eta, \eta \rangle}$ to denote the norm of η .
- We will use the notation $D[\eta, \phi] = \|\eta - \phi\|$ to denote the **distance** between η and ϕ in $\mathcal{L}$

In any linear space $\mathcal{L}$ we have the following two rules:

- **Cauchy-Schwarz:** $|\langle \eta, \phi \rangle| \leq \|\eta\| \|\phi\|$
- **Triangle:** $\|\eta + \phi\| \leq \|\eta\| + \|\phi\|$

Null Spaces, Semi-Inner Products, and Semi-Norms

The **null space** of a non-negative definite bilinear form J in a linear space $\mathcal{L}$ is defined as $\mathcal{N}_J = \{\eta : J(\eta, \eta) = 0, \eta \in \mathcal{L}\}$, and note that

- $\mathcal{N}_J = \{0\}$ if J is positive definite
- $\mathcal{N}_J$ contains 0 and nonzero elements otherwise

A non-negative definite bilinear form J in a linear space $\mathcal{L}$ defines a **semi-inner-product** in $\mathcal{L}$.

- Induces a **semi-norm** $\sqrt{J(\eta)} = \sqrt{J(\eta, \eta)}$ in $\mathcal{L}$
- Similar to a norm, but $J(\eta) = 0$ does not imply $\eta = 0$

Hilbert Spaces and Projections

A **Hilbert space** is a complete inner-product linear space.

- A sequence where $\lim_{m,n \rightarrow \infty} \|\eta_m - \eta_n\| = 0$ is a **Cauchy sequence**
- A linear space $\mathcal{L}$ is **complete** if every Cauchy sequence in $\mathcal{L}$ converges to some element in $\mathcal{L}$.

Any closed linear subspace of $\mathcal{H}$ (denoted $\mathcal{G} \subset \mathcal{H}$) is a Hilbert space.

- Distance between $\eta \in \mathcal{H}$ and $\mathcal{G}$ is $D[\eta, \mathcal{G}] = \inf_{\phi \in \mathcal{G}} \|\eta - \phi\|$
- There exists $\eta_{\mathcal{G}} \in \mathcal{G}$ such that $D[\eta, \mathcal{G}] = \|\eta - \eta_{\mathcal{G}}\|$
- $\eta_{\mathcal{G}}$ is the unique **projection** of η in the space $\mathcal{G}$

Tensor Sum Decompositions

Given $\eta \in \mathcal{H}$ and $\mathcal{G} \subset \mathcal{H}$, we have that $\langle \eta - \eta_{\mathcal{G}}, \phi \rangle = 0$ for all $\phi \in \mathcal{G}$.

- $\mathcal{G}^c = \{\eta : \langle \eta, \phi \rangle = 0, \forall \phi \in \mathcal{G}\}$ is **orthogonal complement** of $\mathcal{G}$
- **Tenor sum decomposition:** $\mathcal{H} = \mathcal{G} \oplus \mathcal{G}^c$ and $\eta = \eta_{\mathcal{G}} + \eta_{\mathcal{G}^c}$

If $\mathcal{H}_n$ and $\mathcal{H}_c$ are Hilbert spaces with inner products $\langle \cdot, \cdot \rangle_n$ and $\langle \cdot, \cdot \rangle_c$, and if $\mathcal{H}_n \cap \mathcal{H}_c = \{0\}$, then $\mathcal{H} = \mathcal{H}_n \oplus \mathcal{H}_c$ is Hilbert space with inner-product $\langle \cdot, \cdot \rangle = \langle \cdot, \cdot \rangle_n + \langle \cdot, \cdot \rangle_c$

Consider a null space $\mathcal{N}_J$ corresponding to a semi-inner-product J in the space $\mathcal{H}$, and define $\tilde{J}(\cdot, \cdot)$ such that

- 1 $\tilde{J}(\cdot, \cdot)$ defines a full inner product in the space $\mathcal{N}_J$
- 2 $(\forall \eta \in \mathcal{H})(\exists \phi \in \mathcal{N}_J)$ such that $\tilde{J}(\eta - \phi) = 0$

Then $(J + \tilde{J})(\eta, \phi)$ defines a full inner product in $\mathcal{H}$.

Hilbert Space Example: $\mathbb{R}^K$

Note that a Hilbert space is a generalization of Euclidean space $\mathbb{R}^K$.

For any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^K$, inner product is $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}'\mathbf{y} = \sum_{i=1}^K x_i y_i$

- $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle_n + \langle \mathbf{x}, \mathbf{y} \rangle_c = \mathbf{x}' \left[\frac{1}{K} \mathbf{1}_K \mathbf{1}_K' + (\mathbf{I}_K - \frac{1}{K} \mathbf{1}_K \mathbf{1}_K') \right] \mathbf{y}$
- $\mathcal{H}_n = \{\eta : \eta(1) = \dots = \eta(K)\}$ and $\mathcal{H}_c = \{\eta : \sum_{x=1}^K \eta(x) = 0\}$

This corresponds to classic one-way ANOVA decomposition

$$\mu_j = \mu + \alpha_j$$

with the constraint $\sum_j \alpha_j = 0$

Riesz Representation Theorem

For every ϕ in a Hilbert space $\mathcal{H}$, the functional $L_\phi \eta = \langle \phi, \eta \rangle$ defines a continuous linear functional L_ϕ .

- L is **continuous** if $\lim_{n \rightarrow \infty} L\eta_n = L\eta$ whenever $\lim_{n \rightarrow \infty} \eta_n = \eta$

Every continuous linear functional L in $\mathcal{H}$ has a representation $L\eta = \langle \phi_L, \eta \rangle$ for some $\phi_L \in \mathcal{H}$, which is called the **representer** of L .

Theorem

For every continuous linear functional L in a Hilbert space $\mathcal{H}$, there exists a unique $\phi_L \in \mathcal{H}$ such that $L\eta = \langle \phi_L, \eta \rangle$ for all $\eta \in \mathcal{H}$.

Reproducing Kernel Hilbert Spaces

To estimate an SSANOVA, we need to evaluate η for different $x \in \mathcal{X}$.

- Need continuity of **evaluation functional**: $[x]\eta = \eta(x)$

Consider a Hilbert space $\mathcal{H}$ of functions on the domain $\mathcal{X}$.

- If evaluation functional $[x]\eta = \eta(x)$ is continuous in $\mathcal{H}$ for all $x \in \mathcal{X}$, then we say that $\mathcal{H}$ is a **reproducing kernel Hilbert space**
- By the Riesz Representation Theorem, there exists $\rho_x \in \mathcal{H}$, which is the representer of the evaluation functional $[x]\eta = \eta(x)$
- Symmetric bivariate function $\rho(x, y) = \rho_x(y) = \langle \rho_x, \rho_y \rangle$ has the reproducing property $\langle \rho(x, \cdot), \eta(\cdot) \rangle = \eta(x)$
- Consequently, ρ is called the **reproducing kernel** of the space $\mathcal{H}$

Examples of Reproducing Kernel Hilbert Spaces

Consider the Euclidean space $\mathbb{R}^K$, which is a RKHS

- Inner product is defined as $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^K x_i y_i$
- RK is define as $\rho(x, y) = I_{\{x=y\}}$, which is indicator function

Consider the space $\mathcal{L}_2[0, 1] = \{\eta : \int_0^1 \eta^2 dx < \infty\}$

- Elements in $\mathcal{L}_2[0, 1]$ are defined via equivalent classes (not defined via individual functions)
- NOT a RKHS because evaluation functional is not well-defined

Consider the space $\mathcal{C}^{(m)}[0, 1] = \{\eta : \eta^{(m)} \in \mathcal{L}_2[0, 1]\}$

- Elements in $\mathcal{C}^{(m)}[0, 1]$ are defined via individual functions
- Evaluation functional is continuous, so we have a RKHS

Tensor Sum Decompositions of RKHS

Given the tensor sum decomposition $\mathcal{H} = \mathcal{H}_n \oplus \mathcal{H}_c$, we have

$$\rho = \rho_n + \rho_c$$

where ρ is the RK of $\mathcal{H}$, ρ_n is the RK of $\mathcal{H}_n$, and ρ_c is the RK of $\mathcal{H}_c$.

Furthermore, if ρ is the RK of $\mathcal{H}$ and if $\rho = \rho_n + \rho_c$ where

- $\rho_n, \rho_c \in \mathcal{H}$ are non-negative for all $x \in \mathcal{X}$
- $\langle \rho_n(x, \cdot), \rho_c(y, \cdot) \rangle = 0$ for all $x, y \in \mathcal{X}$

then the spaces $\mathcal{H}_n$ and $\mathcal{H}_c$ form a tensor sum decomposition of $\mathcal{H}$.

Reproducing Kernel for Nominal Smoothing Splines

Suppose that $x_i \in \mathcal{X} = \{1, \dots, K\}$ and $\eta \in \mathcal{H} = \mathbb{R}^K$.

For any elements $\eta, \phi \in \mathcal{H}$, we have that

- $\langle \eta, \phi \rangle = \boldsymbol{\eta}' \boldsymbol{\phi} = \sum_{x=1}^K \eta(x) \phi(x)$
- $\rho(x, y) = I_{\{x=y\}}$ where $I_{\{\cdot\}}$ is indicator function

Using the averaging operator $A\eta = \sum_{x=1}^K \eta(x)/K$

- $\langle \eta, \phi \rangle = \langle \eta, \phi \rangle_n + \langle \eta, \phi \rangle_c = \boldsymbol{\eta}' \left[\frac{1}{K} \mathbf{1}_K \mathbf{1}_K' + (\mathbf{I}_K - \frac{1}{K} \mathbf{1}_K \mathbf{1}_K') \right] \boldsymbol{\phi}$
- $\rho(x, y) = \rho_n(x, y) + \rho_c(x, y) = \frac{1}{K} + (I_{\{x=y\}} - \frac{1}{K})$
- $\mathcal{H}_n = \{\eta : \eta(1) = \dots = \eta(K)\}$ and $\mathcal{H}_c = \{\eta : \sum_{x=1}^K \eta(x) = 0\}$

Reproducing Kernel for Polynomial Smoothing Splines

Suppose that $x_i \in \mathcal{X} = [0, 1]$ and $\eta \in \mathcal{H} = \mathcal{C}^{(m)}[0, 1]$.

Using the averaging operator $A\eta = \int_0^1 \eta dx$

- $\langle \eta, \phi \rangle = \langle \eta, \phi \rangle_n + \langle \eta, \phi \rangle_c = \sum_{\nu=0}^{m-1} (\int_0^1 \eta^{(\nu)} dx) (\int_0^1 \phi^{(\nu)} dx) + \int_0^1 \eta^{(m)} \phi^{(m)} dx$
- $\rho(x, y) = \rho_n(x, y) + \rho_c(x, y) = \sum_{\nu=0}^{m-1} k_\nu(x) k_\nu(y) + (-1)^{m-1} k_{2m}(|x - y|)$
where $k_\nu(x)$ is scaled Bernoulli polynomial
- $\mathcal{H}_n = \{\eta : \eta^{(m)} = 0\}$ and $\mathcal{H}_c = \{\eta : \int_0^1 \eta^{(\nu)} = 0, \nu = 0, \dots, m-1, \eta^{(m)} \in \mathcal{L}_2[0, 1]\}$

Using the averaging operator $A\eta = \eta(0)$

- $\langle \eta, \phi \rangle = \langle \eta, \phi \rangle_n + \langle \eta, \phi \rangle_c = \sum_{\nu=0}^{m-1} \eta^{(\nu)}(0) \phi^{(\nu)}(0) + \int_0^1 \eta^{(m)} \phi^{(m)} dx$
- $\rho(x, y) = \rho_n(x, y) + \rho_c(x, y) = \sum_{\nu=0}^{m-1} \frac{x^\nu}{\nu!} \frac{y^\nu}{\nu!} + \int_0^1 \frac{(x-u)_+^{m-1}}{(m-1)!} \frac{(y-u)_+^{m-1}}{(m-1)!} du$
- $\mathcal{H}_n = \{\eta : \eta^{(m)} = 0\}$ and $\mathcal{H}_c = \{\eta : \eta^{(\nu)}(0) = 0, \nu = 0, \dots, m-1, \eta^{(m)} \in \mathcal{L}_2[0, 1]\}$

Tensor Product RKHS

Suppose that $\mathbf{x}_i \in \mathcal{X}$ where $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_p$ is a **product domain**.

- Suppose $\mathcal{H}_{\mathcal{X}_j}$ is a RKHS of functions with RK $\rho_{\mathcal{X}_j}$ for all $\mathbf{x}_j \in \mathcal{X}_j$
- Note that the marginal RKs have the form $\rho_{\mathcal{X}_j} = \rho_{\mathbf{n}_j} + \rho_{\mathbf{c}_j}$

We can define $\rho_{\mathcal{X}} = \prod_{j=1}^p \rho_{\mathcal{X}_j} = \prod_{j=1}^p (\rho_{\mathbf{n}_j} + \rho_{\mathbf{c}_j})$

- $\rho_{\mathcal{X}}$ is non-negative for all $\mathbf{x} \in \mathcal{X}$
- $\rho_{\mathcal{X}}$ is RK of **tensor product RKHS** $\mathcal{H} = \mathcal{H}_{\mathcal{X}_1} \otimes \cdots \otimes \mathcal{H}_{\mathcal{X}_p}$

We can form functional spaces for any number of covariates

- Can constrain and/or remove subspaces to fit different models

Need for Additional Smoothing Parameters

Given a tensor product RKHS $\mathcal{H} = \mathcal{H}_{\mathcal{X}_1} \otimes \cdots \otimes \mathcal{H}_{\mathcal{X}_p}$ we have that...

- $\mathcal{H} = \bigotimes_{j=1}^p (\mathcal{H}_{n_j} \oplus \mathcal{H}_{c_j}) = \bigoplus_{k=1}^s \mathcal{H}_k$ is tensor sum decomposition
- Each subspace $\mathcal{H}_k$ has inner product $\langle \cdot, \cdot \rangle_k$ and RK ρ_k
- Inner-products and RKs have different metrics

Can introduce additional smoothing parameters to inner product:

$$\langle \cdot, \cdot \rangle = \sum_{k=1}^s \theta_k^{-1} \langle \cdot, \cdot \rangle_k$$

which corresponds to the tensor product RK

$$\rho = \sum_{k=1}^s \theta_k \rho_k$$

Estimation and Inference

Tensor Product Smoothing Spline

Given $\mathbf{x}_i \in \mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_p$, a **tensor product smoothing spline** is the $\eta_{\lambda} \in \mathcal{H} = \mathcal{H}_{\mathcal{X}_1} \otimes \cdots \otimes \mathcal{H}_{\mathcal{X}_p}$ that minimizes

$$\frac{1}{n} \sum_{i=1}^n (y_i - \eta(\mathbf{x}_i))^2 + \lambda J(\eta)$$

where

- $\lambda \geq 0$ is overall (global) smoothing parameter
- J is a quadratic functional quantifying roughness of η
- Additional smoothing parameters $\boldsymbol{\theta} = (\theta_1, \dots, \theta_s)$ exist in J

Representation of η

Let $\mathcal{H} = \mathcal{H}_n \oplus \mathcal{H}_c$ denote the tensor sum decomposition of the tensor product RKHS $\mathcal{H} = \bigotimes_{j=1}^p \mathcal{H}_{x_j}$

- Note that $\mathcal{H}$ has RK $\rho = \rho_n + \rho_c$ where $\rho_c = \sum_{k=1}^s \theta_k \rho_k$

Given fixed smoothing parameters θ , the $\eta \in \mathcal{H}$ that minimizes the penalized least-squares functional can be written as

$$\eta(\mathbf{x}) = \sum_{v=1}^m d_v \phi_v(\mathbf{x}) + \sum_{i=1}^n c_i \rho_c(\mathbf{x}_i, \mathbf{x}) \quad (1)$$

where $\{\phi_v\}_{v=1}^m$ is a set of known functions spanning $\mathcal{H}_n$, ρ_c is the reproducing kernel (RK) of $\mathcal{H}_c$, and $\mathbf{d} \equiv \{d_v\}_{m \times 1}$ and $\mathbf{c} \equiv \{c_i\}_{n \times 1}$ are the (unknown) basis function coefficient vectors

Penalty of η

Given the tensor sum decomposition $\mathcal{H} = \mathcal{H}_n \oplus \mathcal{H}_c$ for some tensor product RKHS $\mathcal{H} = \bigotimes_{j=1}^p \mathcal{H}_{x_j}$, we define the penalty functional

$$J(\eta) = \langle \eta, \eta \rangle_c$$

which is a semi-inner-product with null space $\mathcal{H}_n$.

Using the representation of $\eta(\mathbf{x})$ on the previous slide, we have

$$\begin{aligned} \langle \eta, \eta \rangle_c &= \langle \sum_{v=1}^m d_v \phi_v(\mathbf{x}) + \sum_{i=1}^n c_i \rho_c(\mathbf{x}_i, \mathbf{x}), \sum_{v=1}^m d_v \phi_v(\mathbf{x}) + \sum_{i=1}^n c_i \rho_c(\mathbf{x}_i, \mathbf{x}) \rangle_c \\ &= \langle \sum_{i=1}^n c_i \rho_c(\mathbf{x}_i, \mathbf{x}), \sum_{i=1}^n c_i \rho_c(\mathbf{x}_i, \mathbf{x}) \rangle_c \\ &= \sum_{i=1}^n \sum_{j=1}^n c_i c_j \langle \rho_c(\mathbf{x}_i, \mathbf{x}), \rho_c(\mathbf{x}_j, \mathbf{x}) \rangle_c = \sum_{i=1}^n \sum_{j=1}^n c_i c_j \rho_c(\mathbf{x}_i, \mathbf{x}_j) \end{aligned}$$

Penalized Least Squares Problem

Using $\{\mathbf{x}_u^*\}_{u=1}^q \subset \{\mathbf{x}_i\}_{i=1}^n$ as knots, the penalized least-squares functional can be approximated as

$$\|\mathbf{y} - \mathbf{K}\mathbf{d} - \mathbf{J}_\theta\mathbf{c}\|^2 + \lambda n\mathbf{c}'\mathbf{Q}_\theta\mathbf{c}$$

where

- $\mathbf{y} = (y_1, \dots, y_n)'$ is response vector
- $\mathbf{K} = \{\phi_v(\mathbf{x}_i)\}_{n \times m}$ is null space basis function matrix
- $\mathbf{J}_\theta = \{\rho_c(\mathbf{x}_i, \mathbf{x}_u^*)\}_{n \times q}$ is contrast space basis function matrix

Note: $\mathbf{J}_\theta = \sum_{k=1}^S \theta_k \mathbf{J}_k$ where $\mathbf{J}_k = \{\rho_k(\mathbf{x}_i, \mathbf{x}_u^*)\}_{n \times q}$

- $\mathbf{Q}_\theta = \{\rho_c(\mathbf{x}_t^*, \mathbf{x}_u^*)\}_{q \times q}$ is penalty matrix

Note: $\mathbf{Q}_\theta = \sum_{k=1}^S \theta_k \mathbf{Q}_k$ where $\mathbf{Q}_k = \{\rho_k(\mathbf{x}_t^*, \mathbf{x}_u^*)\}_{q \times q}$

- $\mathbf{d} = (d_1, \dots, d_m)'$ and $\mathbf{c} = (c_1, \dots, c_q)'$ are unknown coefficients

Coefficients and Smoothing Matrix

The coefficients minimizing the penalized least-squares function are

$$\begin{pmatrix} \hat{\mathbf{d}} \\ \hat{\mathbf{c}} \end{pmatrix} = \begin{pmatrix} \mathbf{K}'\mathbf{K} & \mathbf{K}'\mathbf{J}_\theta \\ \mathbf{J}_\theta'\mathbf{K} & \mathbf{J}_\theta'\mathbf{J}_\theta + \lambda n\mathbf{Q}_\theta \end{pmatrix}^\dagger \begin{pmatrix} \mathbf{K}' \\ \mathbf{J}_\theta' \end{pmatrix} \mathbf{y}$$

where $(\cdot)^\dagger$ denotes the Moore-Penrose pseudoinverse.

The fitted values are given by $\hat{\mathbf{y}} = \mathbf{K}\hat{\mathbf{d}} + \mathbf{J}_\theta\hat{\mathbf{c}} = \mathbf{S}_\lambda\mathbf{y}$ where

$$\mathbf{S}_\lambda = (\mathbf{K} \quad \mathbf{J}_\theta) \begin{pmatrix} \mathbf{K}'\mathbf{K} & \mathbf{K}'\mathbf{J}_\theta \\ \mathbf{J}_\theta'\mathbf{K} & \mathbf{J}_\theta'\mathbf{J}_\theta + \lambda n\mathbf{Q}_\theta \end{pmatrix}^\dagger \begin{pmatrix} \mathbf{K}' \\ \mathbf{J}_\theta' \end{pmatrix}$$

is the **smoothing matrix**, which depends on $\lambda = (\lambda/\theta_1, \dots, \lambda/\theta_s)$.

Smoothing Parameter Goldilocks Phenomenon

The selection of λ is the crucial step when fitting an SSANOVA model!

If $\lambda_k \equiv \lambda/\theta_k$ is too large, the penalty corresponding to $\mathcal{H}_k$ will be too severe, making it difficult to estimate η_k .

- Oversmooth k -th contrast space

If $\lambda_k \equiv \lambda/\theta_k$ is too small, the penalty corresponding to $\mathcal{H}_k$ will be too lenient, making it difficult to estimate η_k (assuming noisy data)

- Undersmooth k -th contrast space

Cross-Validation

If σ^2 is unknown, a reasonable loss function for selecting λ is the cross-validated loss function

$$\text{CV}(\lambda|\mathbf{y}, \mathbf{X}, \mathbf{w}) = (1/n) \sum_{i=1}^n w_i (y_i - \eta_{\lambda}^{[i]}(\mathbf{x}_i))^2$$

where $w_i > 0$ is some weight, and $\eta_{\lambda}^{[i]}$ is the function $\phi \in \mathcal{H}$ that minimizes the **delete the i -th observation functional**:

$$(1/n) \sum_{j \neq i} (y_j - \phi(\mathbf{x}_j))^2 + \lambda J(\phi)$$

Cross-Validation (continued)

The form of CV loss function might suggest that it is necessary to fit n different models (to obtain $\eta_{\lambda}^{[i]}$ for $i \in \{1, \dots, n\}$).

However, the CV function can be rewritten as

$$\text{CV}(\lambda | \mathbf{y}, \mathbf{X}, \mathbf{w}) = (1/n) \sum_{i=1}^n w_i \frac{(y_i - \eta_{\lambda}(\mathbf{x}_i))^2}{(1 - s_{ii}(\lambda))^2}$$

which implies that the CV function can be minimized using the results of the full model.

Generalized Cross-Validation

Defining $w_i \equiv (1 - s_{ii}(\lambda))^2 / [n^{-1} \text{tr}(\mathbf{I}_n - \mathbf{S}_\lambda)]^2$ replaces each $s_{ii}(\lambda)$ with its average value, producing the generalized cross-validation (GCV) criterion of Craven and Wahba (1979):

$$\begin{aligned} \text{GCV}(\lambda | \mathbf{y}, \mathbf{X}) &= (1/n) \sum_{i=1}^n \frac{(y_i - \eta_\lambda(\mathbf{x}_i))^2}{[n^{-1} \text{tr}(\mathbf{I}_n - \mathbf{S}_\lambda)]^2} \\ &= \frac{(1/n) \|(\mathbf{I}_n - \mathbf{S}_\lambda) \mathbf{y}\|^2}{[1 - \text{tr}(\mathbf{S}_\lambda)/n]^2} \end{aligned} \quad (2)$$

The $\hat{\lambda}$ that minimizes the GCV score produces good estimates of η .

Gaussian Process Definition

A **Gaussian process** is a stochastic process $\{\eta(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ such that $\eta(\mathbf{x}) \sim N(\mu_x, \sigma_x^2)$ for all $\mathbf{x} \in \mathcal{X}$ where

- $\mu_x = E(\eta(\mathbf{x}))$ is the mean function
- $\gamma_{x,x'} = \text{Cov}(\eta(\mathbf{x}), \eta(\mathbf{x}'))$ is the covariance function
- $\sigma_x^2 = \text{Cov}(\eta(\mathbf{x}), \eta(\mathbf{x}))$ is the variance function

Note $\eta(\mathbf{x})$ is a random variable that is normally distributed for all $\mathbf{x} \in \mathcal{X}$

- Use the notation $\eta(\mathbf{x}) \sim N(\mu_x, \sigma_x^2)$ for all $\mathbf{x} \in \mathcal{X}$
- Mean and variance differ for each $\mathbf{x} \in \mathcal{X}$

Bayesian Interpretation of Smoothing Spline

Let $\eta = \eta_n + \eta_c$ denote the null and contrast space functions and assume the following prior distributions:

- η_n has a diffuse (vague) prior with mean zero
- η_c is a zero mean Gaussian process with covariance function proportional to ρ_c

Using these prior assumptions. . .

- $\hat{\eta}$ can be interpreted as posterior mean of η given data $\mathbf{y}$
- we can derive posterior variance $\text{Var}(\eta|\mathbf{y})$

Bayesian Confidence Intervals

Using the Bayesian interpretation, we can form confidence intervals

$$\hat{\eta}(\mathbf{x}) \pm Z_{\alpha/2} \sqrt{\text{Var}(\eta|\mathbf{y})}$$

where $Z_{\alpha/2}$ is critical value from standard normal distribution.

Bayesian CIs have approximate “across-the-function coverage” when the smoothing parameters are selected according to GCV.

- On average contain $100(1 - \alpha)\%$ of true function realizations

SSANOVA in Practice

Unidimensional Smoothing Splines in R

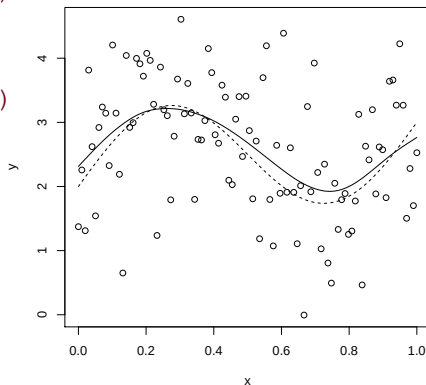
Many options for unidimensional smoothing spline in R:

- `smooth.spline` function (in `stats` package)
- `bigsspline` function (in `bigsplines` package)
- `bigssa` function (in `bigsplines` package)
- `ssanova` function (in `gss` package)
- `gam` function (in `mgcv` package)

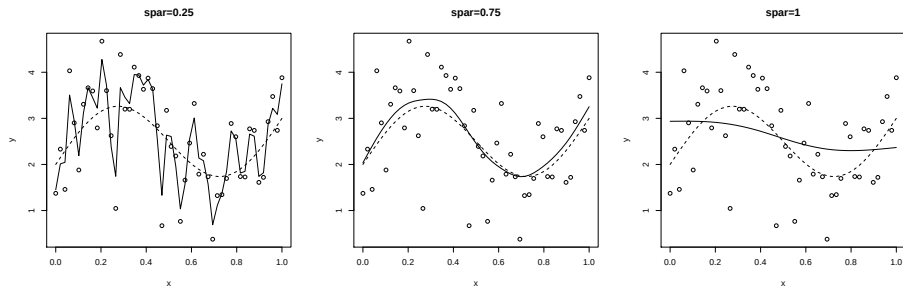
For unidimensional smoothing, we will focus on the `smooth.spline` and the `bigsspline` functions, which have simple syntax.

smooth.spline: Overview

```
> set.seed(1)
> x = seq(0,1,length=100)
> eta = 2 + x + sin(2*pi*x)
> y = eta + rnorm(100)
> plot(x,y)
> smsp = smooth.spline(x,y)
> lines(x,smsp$y)
> lines(x,eta,lty=2)
```



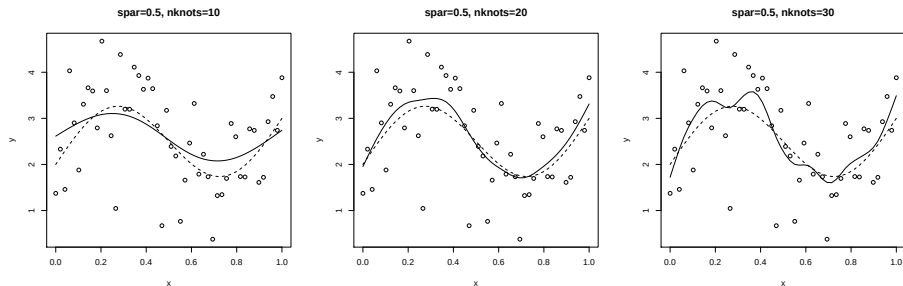
smooth.spline: Changing Smoothing Parameter



R code for leftmost plot:

```
> smsp = smooth.spline(x,y,spar=0.25)
> plot(x,y,main="spar=0.25")
> lines(x,smsp$y)
> lines(x,eta,lty=2)
```

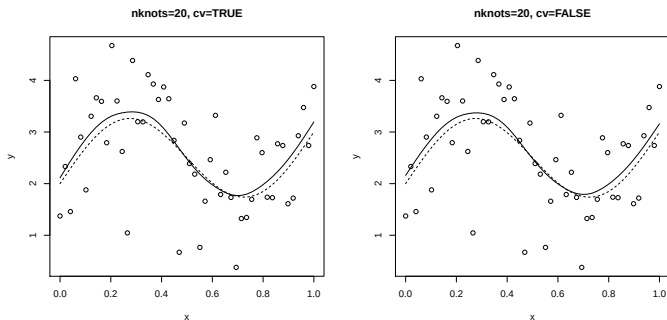
smooth.spline: Changing Number of Knots



R code for leftmost plot:

```
> smsp = smooth.spline(x,y,spar=0.5,nknots=10)
> plot(x,y,main="spar=0.5, nknots=10")
> lines(x,smsp$y)
> lines(x,eta,lty=2)
```

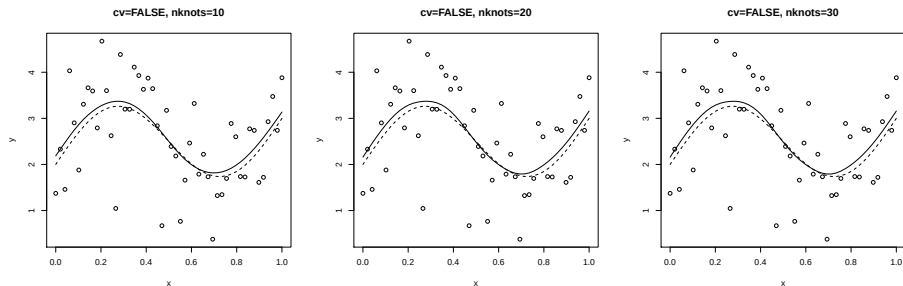
smooth.spline: CV versus GCV



R code for leftmost plot:

```
> smsp = smooth.spline(x,y,nknots=20,cv=TRUE)
> plot(x,y,main="nknots=20, cv=TRUE")
> lines(x,smsp$y)
> lines(x,eta,lty=2)
```

smooth.spline: Number of Knots (revisited)



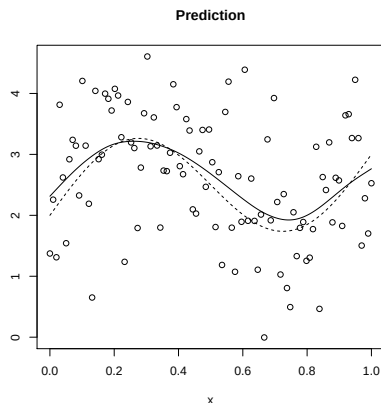
R code for leftmost plot:

```
> smsp = smooth.spline(x,y,nknots=10)
> plot(x,y,main="cv=FALSE, nknots=10")
> lines(x,smsp$y)
> lines(x,eta,lty=2)
```

smooth.spline: Predicting for New Data

Given $\hat{\eta}$ we can predict for a new sequence of data:

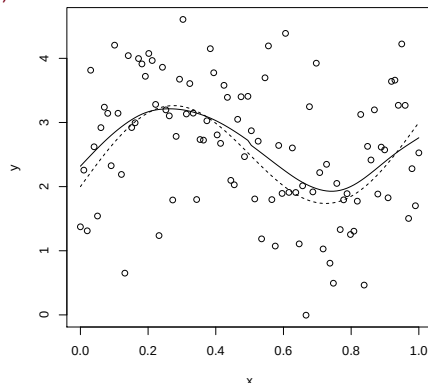
```
> set.seed(1)
> x = seq(0,1,length=100)
> eta = 2 + x + sin(2*pi*x)
> y = eta + rnorm(100)
> plot(x,y,main="Prediction")
> smsp = smooth.spline(x,y)
> newdata = seq(0,1,length=200)
> yhat = predict(smsp,newdata)
> lines(yhat)
> lines(x,eta,lty=2)
```



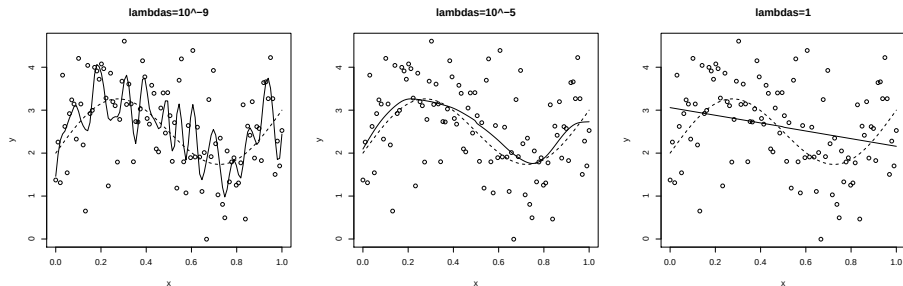
bigspline: Overview

For smoothing large samples...

```
> set.seed(1)
> x = seq(0,1,length=100)
> eta = 2 + x + sin(2*pi*x)
> y = eta + rnorm(100)
> plot(x,y)
> bigsp = bigspline(x,y)
> lines(x,bigsp$fitted)
> lines(x,eta,lty=2)
```



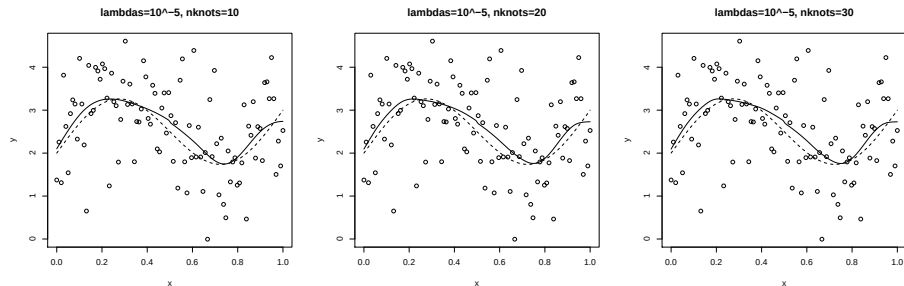
bigspline: Changing Smoothing Parameter



R code for leftmost plot:

```
> bigsp = bigspine(x,y,lambda=10^-9)
> plot(x,y,main="lambda=10^-9")
> lines(x,bigsp$fitted)
> lines(x,eta,lty=2)
```

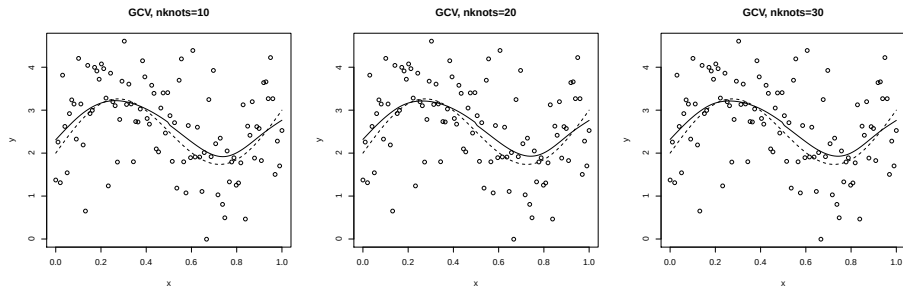
bigspline: Changing Number of Knots



R code for leftmost plot:

```
> bigsp = bigspline(x,y,lambdas=10^-5,nknots=10)
> plot(x,y,main="lambdas=10^-5, nknots=10")
> lines(x,bigsp$fitted)
> lines(x,eta,lty=2)
```

bigspline: Number of Knots (revisited)



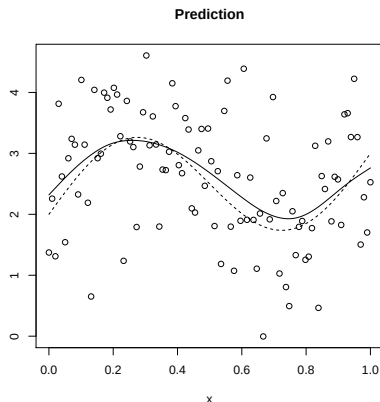
R code for leftmost plot:

```
> bigsp = bigspline(x,y,nknots=10)
> plot(x,y,main="GCV, nknots=10")
> lines(x,bigsp$fitted)
> lines(x,eta,lty=2)
```

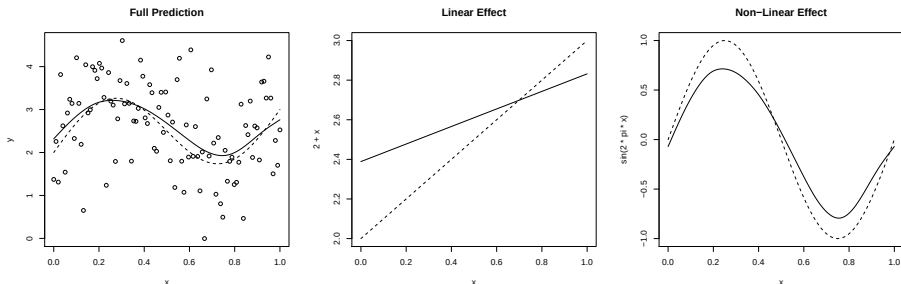
bigspline: Predicting for New Data

Given $\hat{\eta}$ we can predict for a new sequence of data:

```
> set.seed(1)
> x = seq(0,1,length=100)
> eta = 2 + x + sin(2*pi*x)
> y = eta + rnorm(100)
> plot(x,y,main="Prediction")
> bigsp = bigspline(x,y)
> newdata = seq(0,1,length=200)
> yhat = predict(bigsp,newdata)
> lines(newdata,yhat)
> lines(x,eta,lty=2)
```



bigspline: Predicting Linear and Non-Linear Effects

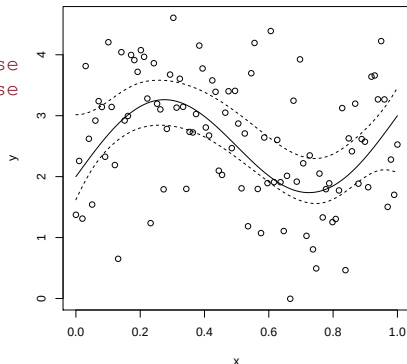


R code for center and rightmost plot:

```
> newdata = seq(0,1,length=200)
> plot(x,2+x,main="Linear Effect",type="l",lty=2)
> yhat = predict(bigsp,newdata,effect="0") + predict(bigsp,newdata,effect="lin")
> lines(newdata,yhat)
> plot(x,sin(2*pi*x),main="Non-Linear Effect",type="l",lty=2)
> yhat = predict(bigsp,newdata,effect="non")
> lines(newdata,yhat)
```

bigspline: Bayesian Confidence Intervals

```
> dev.new(width=6,height=6,noRStudioGD=TRUE)
> set.seed(1)
> x = seq(0,1,length=100)
> eta = 2 + x + sin(2*pi*x)
> y = eta + rnorm(100)
> bigsp = bigspline(x,y,se.fit=TRUE)
> cilo = bigsp$fit - qnorm(0.975)*bigsp$se
> cihi = bigsp$fit + qnorm(0.975)*bigsp$se
> plot(x,y)
> lines(x,eta)
> lines(bigsp$xunique,cilo,lty=2)
> lines(bigsp$xunique,cihi,lty=2)
> sum(eta>=cilo & eta<=cihi)/length(x)
[1] 1
```



Comparing `smooth.spline` and `bigspline`

Consider the function $y_i = 2 + x_i + \sin(2\pi x_i) + e_i$ where $e_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2)$.

Suppose that $x_i = i/n$ for $i \in \{0, \dots, n\}$ and $\sigma^2 = 1$ so that $e_i \stackrel{\text{iid}}{\sim} N(0, 1)$.

Median true MSE = $\frac{1}{n} \sum_{i=1}^n (\eta(x_i) - \hat{\eta}(x_i))^2$ using $q = 20$ knots:

n	100	1000	10000	1e+05	1e+06
<code>smooth.spline</code>	0.13836	0.00504	0.00113	1e-04	2e-05
<code>bigspline</code>	0.14030	0.00497	0.00110	1e-04	2e-05

Median runtimes using $q = 20$ knots:

n	100	1000	10000	1e+05	1e+06
<code>smooth.spline</code>	0.001	0.002	0.021	0.1965	2.233
<code>bigspline</code>	0.009	0.009	0.011	0.0120	0.094

R Code for Simulation (on previous slide)

```
nsamp = 10^c(2:6)
simresults = NULL
xnew = seq(0,1,length=200)
set.seed(1)
for(j in 1:5){
  for(k in 1:10){
    x = seq(0,1,length=nsamp[j])
    eta = 2 + x + sin(2*pi*x)
    y = eta + rnorm(nsamp[j])

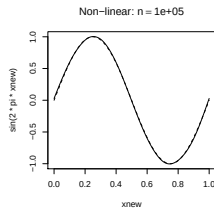
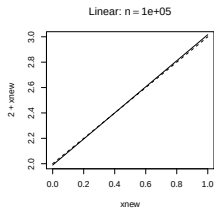
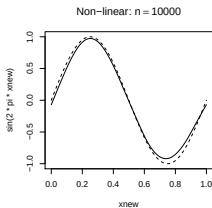
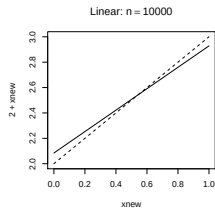
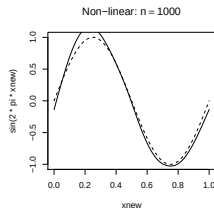
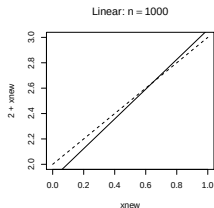
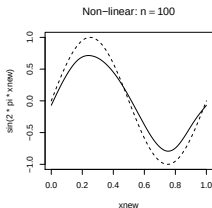
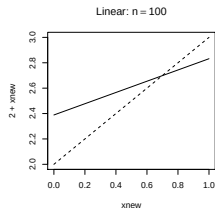
    tic = proc.time()
    ssmod = smooth.spline(x,y,nknots=20)
    toc = proc.time() - tic
    tmse = sum( (ssmod$y - eta)^2 ) / nsamp[j]
    simsp = data.frame(method="smsp",n=nsamp[j],time=toc[3],tmse=tmse,row.names=k)

    tic = proc.time()
    ssmod = bigspline(x,y,nknots=20)
    toc = proc.time() - tic
    tmse = sum( (predict(ssmod) - eta)^2 ) / nsamp[j]
    simbig = data.frame(method="big",n=nsamp[j],time=toc[3],tmse=tmse,row.names=k+1)

    simresults = rbind(simresults,simsp,simbig)
  }
}

round(tapply(simresults$tmse,list(simresults$method,simresults$n),median),5)
round(tapply(simresults$time,list(simresults$method,simresults$n),median),5)
```

big spline: Linear and Non-Linear Effects (revisited)



Multidimensional Smoothing Splines in R

A few options for multidimensional smoothing splines in R:

- `bigssa` function (in `bigsplines` package)
- `bigssp` function (in `bigsplines` package)
- `ssanova` function (in `gss` package)
- `gam` function (in `mgcv` package)

We will focus on the `ssanova` and `bigssa` (or `bigssp`) functions, which fit tensor product smoothing splines.

Note that the `gam` function handles interactions in different manner.

Additive Function: Definition

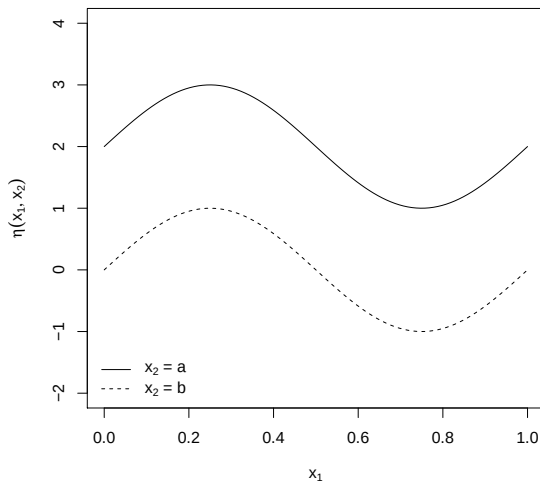
Suppose we have the following function defined for $\mathbf{x} = (x_1, x_2) \in [0, 1] \times \{a, b\}$:

```
addfun = function(x1,x2){  
  funval = sin(2*pi*x1)  
  idx = which(x2=="a")  
  funval[idx] = funval[idx] + 2  
  funval  
}
```

Note that the function is

- $\eta(x_1, x_2) = 2 + \sin(2\pi x_1)$ if $x_2 = a$
- $\eta(x_1, x_2) = \sin(2\pi x_1)$ if $x_2 \neq a$

Additive Function: Visualization



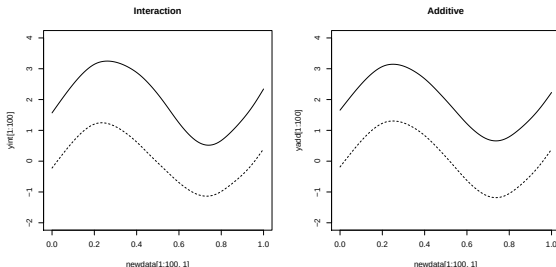
Additive Function: `bigssa` fitting

```
> n = 100
> set.seed(55455)
> x1v = seq(0,1,length=n)
> x2v = factor(sample(letters[1:2],n,replace=TRUE))
> eta = addfun(x1v,x2v)
> y = eta + rnorm(n)
> idx = binsamp(cbind(x1v,x2v),nmbin=c(20,2))
> ssint = bigssa(y~x1v*x2v, type=list(x1v="cub",x2v="nom"), nknots=idx)
> sum((ssint$fitted-eta)^2) / length(eta)
[1] 0.04605668
> ssadd = bigssa(y~x1v+x2v, type=list(x1v="cub",x2v="nom"), nknots=idx)
> sum((ssadd$fitted-eta)^2) / length(eta)
[1] 0.03305623
> fitstats = rbind(ssint$info,ssadd$info)
> rownames(fitstats) = c("int","add")
> fitstats
```

	gcv	rsq	aic	bic
int	1.441561	0.6258559	319.3529	344.6341
add	1.386159	0.6134636	316.0127	332.6986

Additive Function: bigssa prediction

```
> dev.new(width=12,height=6,noRStudioGD=TRUE)
> par(mfrow=c(1,2))
> newdata = expand.grid(x1v=seq(0,1,length=100),x2v=c("a","b"))
> yint = predict(ssint,newdata)
> yadd = predict(ssadd,newdata)
> plot(newdata[1:100,1],yint[1:100],main="Interaction",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yint[101:200],lty=2)
> plot(newdata[1:100,1],yadd[1:100],main="Additive",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yadd[101:200],lty=2)
```

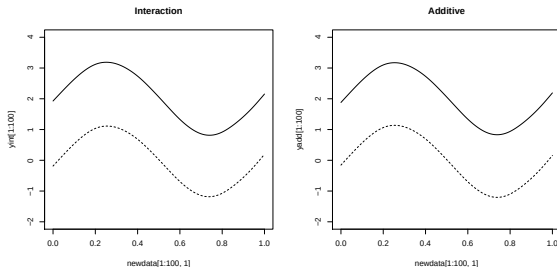


Additive Function: `ssanova` fitting

```
> n = 100
> set.seed(55455)
> x1v = seq(0,1,length=n)
> x2v = factor(sample(letters[1:2],n,replace=TRUE))
> eta = addfun(x1v,x2v)
> y = eta + rnorm(n)
> idx = binsamp(cbind(x1v,x2v),nmbin=c(20,2))
> ssint = ssanova(y~x1v*x2v,type=list(x1v="cubic",x2v="nominal"),
+               id.basis=idx)
> newdata = data.frame(x1v=x1v,x2v=x2v)
> sum((predict(ssint,newdata)-eta)^2) / length(eta)
[1] 0.01449173
> ssadd = ssanova(y~x1v+x2v,type=list(x1v="cubic",x2v="nominal"),
+               id.basis=idx)
> sum((predict(ssadd,newdata)-eta)^2) / length(eta)
[1] 0.01432404
```

Additive Function: `ssanova` prediction

```
> dev.new(width=12,height=6,noRStudioGD=TRUE)
> par(mfrow=c(1,2))
> newdata = expand.grid(x1v=seq(0,1,length=100),x2v=c("a","b"))
> yint = predict(ssint,newdata)
> yadd = predict(ssadd,newdata)
> plot(newdata[1:100,1],yint[1:100],main="Interaction",
+      type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yint[101:200],lty=2)
> plot(newdata[1:100,1],yadd[1:100],main="Additive",
+      type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yadd[101:200],lty=2)
```



Interaction Function: Definition

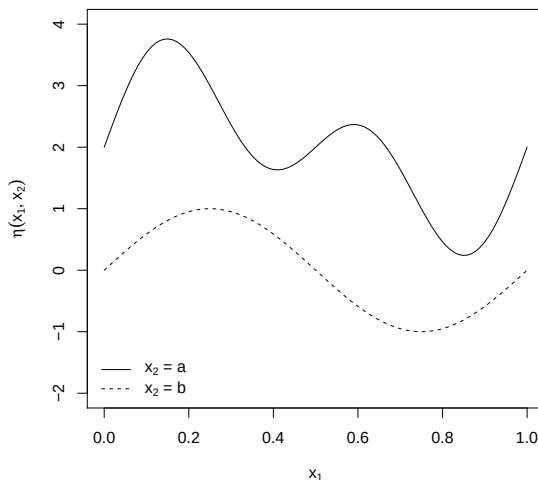
Suppose we have the following function defined for $\mathbf{x} = (x_1, x_2) \in [0, 1] \times \{a, b\}$:

```
intfun = function(x1,x2){  
  funval = sin(2*pi*x1)  
  idx = which(x2=="a")  
  funval[idx] = funval[idx] + 2 + sin(4*pi*x1[idx])  
  funval  
}
```

Note that the function is

- $\eta(x_1, x_2) = 2 + \sin(2\pi x_1) + \sin(4\pi x_1)$ if $x_2 = a$
- $\eta(x_1, x_2) = \sin(2\pi x_1)$ if $x_2 \neq a$

Interaction Function: Visualization



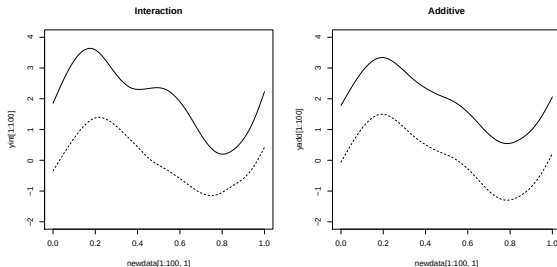
Interaction Function: `bigssa` fitting

```
> n = 100
> set.seed(55455)
> x1v = seq(0,1,length=n)
> x2v = factor(sample(letters[1:2],n,replace=TRUE))
> eta = intfun(x1v,x2v)
> y = eta + rnorm(n)
> idx = binsamp(cbind(x1v,x2v),nmbin=c(20,2))
> ssint = bigssa(y~x1v*x2v,type=list(x1v="cub",x2v="nom"),nknots=idx)
> sum((ssint$fitted-eta)^2) / length(eta)
[1] 0.1081747
> ssadd = bigssa(y~x1v+x2v,type=list(x1v="cub",x2v="nom"),nknots=idx)
> sum((ssadd$fitted-eta)^2) / length(eta)
[1] 0.1858098
> fitstats = rbind(ssint$info,ssadd$info)
> rownames(fitstats) = c("int","add")
> fitstats
```

	gcv	rsq	aic	bic
int	1.522061	0.6509204	324.0861	356.6680
add	1.510616	0.6097741	324.5034	343.1147

Interaction Function: `bigssa` prediction

```
> dev.new(width=12,height=6,noRStudioGD=TRUE)
> par(mfrow=c(1,2))
> newdata = expand.grid(x1v=seq(0,1,length=100),x2v=c("a","b"))
> yint = predict(ssint,newdata)
> yadd = predict(ssadd,newdata)
> plot(newdata[1:100,1],yint[1:100],main="Interaction",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yint[101:200],lty=2)
> plot(newdata[1:100,1],yadd[1:100],main="Additive",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yadd[101:200],lty=2)
```

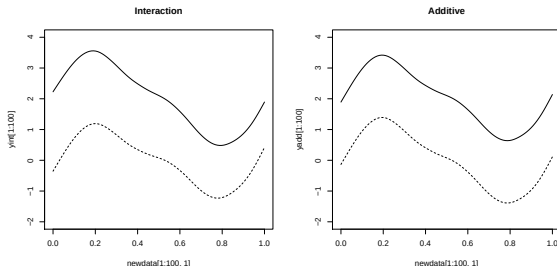


Interaction Function: `ssanova` fitting

```
> n = 100
> set.seed(55455)
> x1v = seq(0,1,length=n)
> x2v = factor(sample(letters[1:2],n,replace=TRUE))
> eta = intfun(x1v,x2v)
> y = eta + rnorm(n)
> idx = binsamp(cbind(x1v,x2v),nmbin=c(20,2))
> ssint = ssanova(y~x1v*x2v,type=list(x1v="cubic",x2v="nominal"),
+               id.basis=idx)
> newdata = data.frame(x1v=x1v,x2v=x2v)
> sum((predict(ssint,newdata)-eta)^2) / length(eta)
[1] 0.1624814
> ssadd = ssanova(y~x1v+x2v,type=list(x1v="cubic",x2v="nominal"),
+               id.basis=idx)
> sum((predict(ssadd,newdata)-eta)^2) / length(eta)
[1] 0.1802812
```

Interaction Function: `ssanova` prediction

```
> dev.new(width=12,height=6,noRStudioGD=TRUE)
> par(mfrow=c(1,2))
> newdata = expand.grid(x1v=seq(0,1,length=100),x2v=c("a","b"))
> yint = predict(ssint,newdata)
> yadd = predict(ssadd,newdata)
> plot(newdata[1:100,1],yint[1:100],main="Interaction",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yint[101:200],lty=2)
> plot(newdata[1:100,1],yadd[1:100],main="Additive",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yadd[101:200],lty=2)
```



Interaction Function: Fitting with More Data

```
> n = 1000
> set.seed(55455)
> x1v = seq(0,1,length=n)
> x2v = factor(sample(letters[1:2],n,replace=TRUE))
> eta = intfun(x1v,x2v)
> y = eta + rnorm(n)
> idx = binsamp(cbind(x1v,x2v),nmbin=c(20,2))
> ssint = bigssa(y~x1v*x2v,type=list(x1v="cub",x2v="nom"),nknots=idx)
> sum((ssint$fitted-eta)^2) / length(eta)
[1] 0.03178251
> ssadd = bigssa(y~x1v+x2v,type=list(x1v="cub",x2v="nom"),nknots=idx)
> sum((ssadd$fitted-eta)^2) / length(eta)
[1] 0.1311356
> fitstats = rbind(ssint$info,ssadd$info)
> rownames(fitstats) = c("int","add")
> fitstats
```

	gcv	rsq	aic	bic
int	1.016522	0.6479397	2854.060	2923.757
add	1.081167	0.6236793	2915.779	2973.402

Interaction Function: Predicting with More Data

```
> dev.new(width=12,height=6,noRStudioGD=TRUE)
> par(mfrow=c(1,2))
> newdata = expand.grid(x1v=seq(0,1,length=100),x2v=c("a","b"))
> yint = predict(ssint,newdata)
> yadd = predict(ssadd,newdata)
> plot(newdata[1:100,1],yint[1:100],main="Interaction",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yint[101:200],lty=2)
> plot(newdata[1:100,1],yadd[1:100],main="Additive",
+       type="l",ylim=c(-2,4))
> lines(newdata[101:200,1],yadd[101:200],lty=2)
```

